Programming Assignment Report

Design and Analysis of Algorithms

Name (as in ERP): Shobith IK

Registration Number : 2411021061604

Problem Name: Real Estate Broker

Problem Statement:
Find the maximum number of sales by matching clients to houses where the house area is strictly greater than a_i and the price is less than or equal to p_i.

Problem HackerRank Link: https://www.hackerrank.com/challenges/real-estate-broker/problem

Github Submission Link: https://github.com/Shobith15/DAA-Assignment/tree/main

---

1. Problem Explanation

You are given m unsold houses, where each house j has an area x_j and a minimum price y_j. There are also n clients, and each client i wants a house with an area greater than a_i and a price less than or equal to p_i.

The objective is to:

- Match clients to available houses based on their specific area and price requirements.
- Adhere to the constraint that each client can buy at most one house and each house can have at most one owner.
- Determine the maximum number of houses you can sell.

---

2. Example

Input: * Client 0: wants area > 5, price <= 110

- Client 1: wants area > 9, price <= 500
- House 0: area = 10, price = 100
- House 1: area = 30, price = 300

Valid Pairs: * Client 0 can buy House 0 (10 > 5 and 100 <= 110).

- Client 1 can buy House 1 (30 > 9 and 300 <= 500).

Maximum Count = 2

---

3. **Approach / Algorithm**

   Step 1: Graph Modeling Represent the problem as a Bipartite Graph. Create two sets of nodes: one for clients and one for houses. An edge exists between a client and a house if the house meets the client's criteria.

   Step 2: Maximum Bipartite Matching Use a Depth-First Search (DFS) based algorithm to find augmenting paths. This identifies the highest number of unique pairings possible.

   Step 3: Optimization By attempting to match each client sequentially and re-routing existing matches when a conflict occurs, we find the global maximum matching.

---

4. **Time Complexity**

   5. **Adjacency Construction:** O(n . m)
   6. **DFS Matching:** O(n . m)
   7. **Total Complexity:** O(n . m), which is efficient for n, m <= 1000.

Space Complexity * **Storage:** O(n . m) for the adjacency list and matching arrays.

---

5. **Complete Java Code**

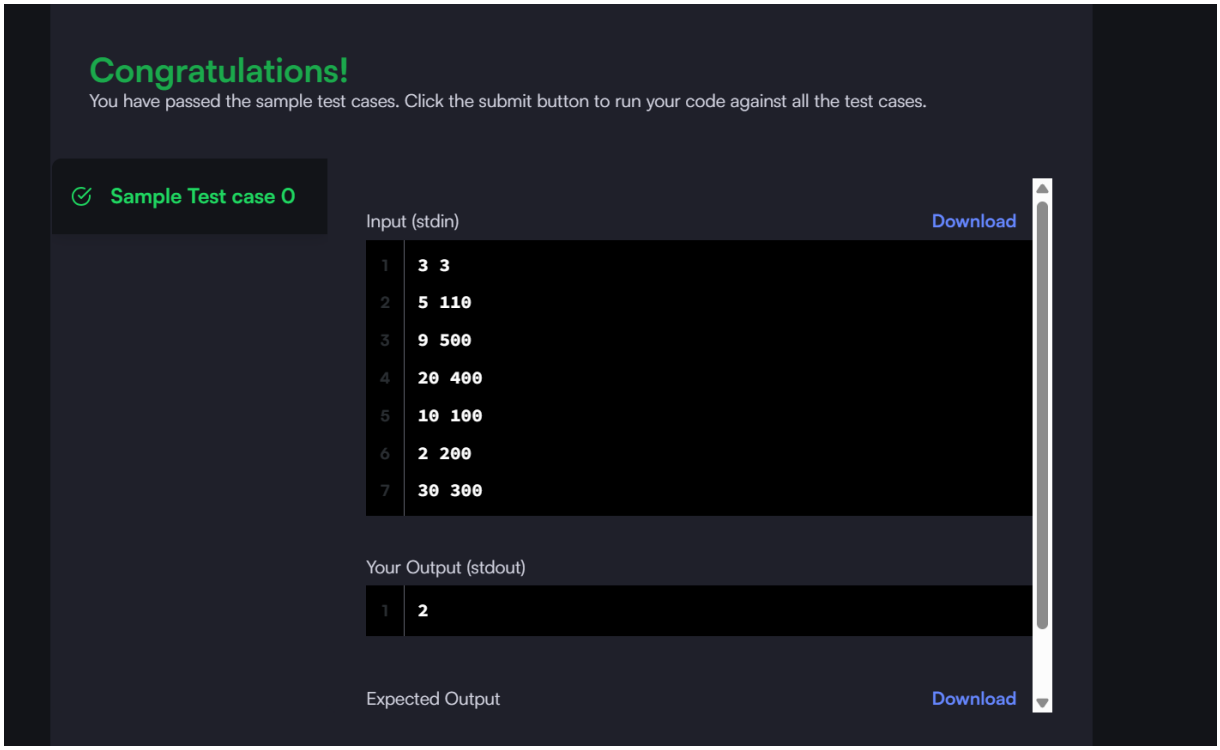
```
import java.util.*;


public class Solution {

   private static boolean canMatch(int client, List<Integer>[] adj, int[] match, boolean[]
visited) {

      for (int house : adj[client]) {

         if (!visited[house]) {

            visited[house] = true;

            if (match[house] < 0 || canMatch(match[house], adj, match, visited)) {

               match[house] = client;

               return true;

            }

         }

      }
```

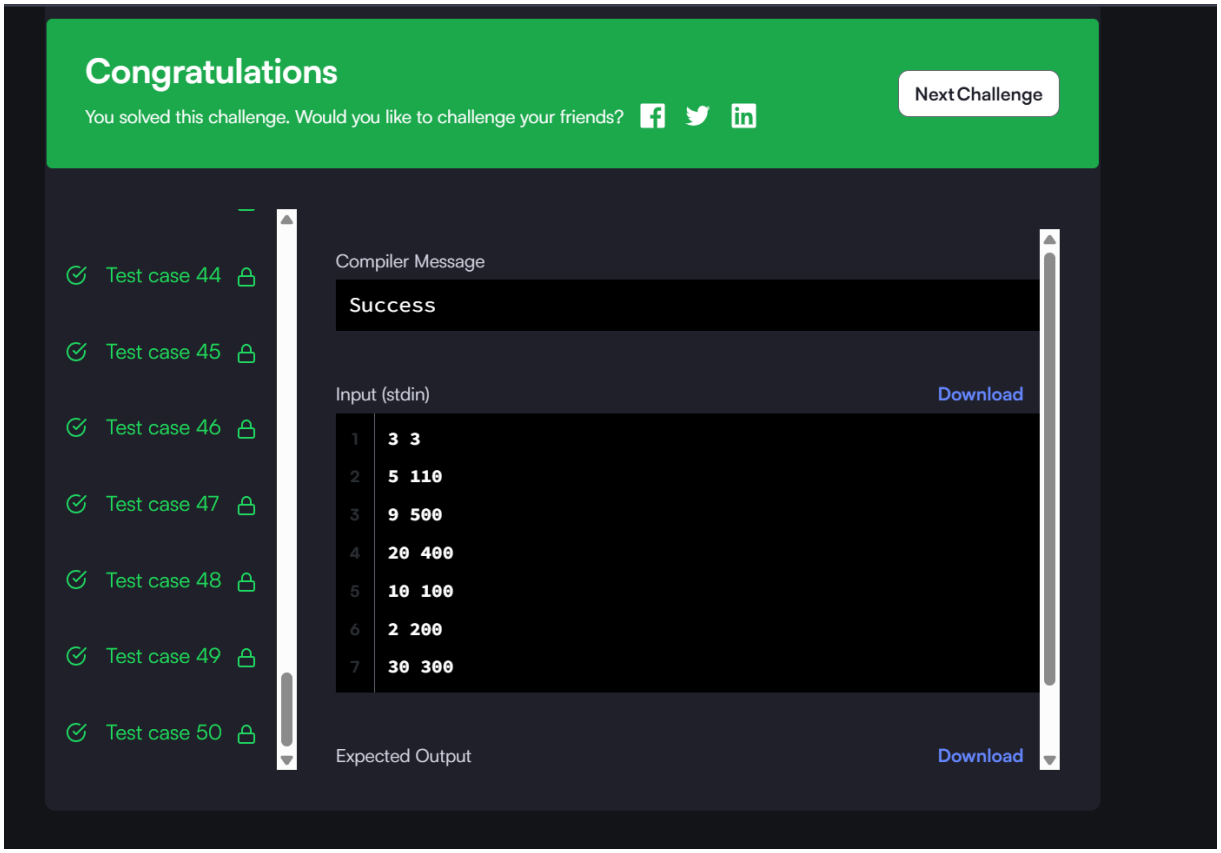
```
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        if (!sc.hasNextInt()) return;
        int n = sc.nextInt();
        int m = sc.nextInt();

        long[][] clients = new long[n][2];
        for (int i = 0; i < n; i++) {
            clients[i][0] = sc.nextLong();
            clients[i][1] = sc.nextLong();
        }

        long[][] houses = new long[m][2];
        for (int j = 0; j < m; j++) {
            houses[j][0] = sc.nextLong();
            houses[j][1] = sc.nextLong();
        }

        List<Integer>[] adj = new ArrayList[n];
        for (int i = 0; i < n; i++) {
            adj[i] = new ArrayList<>();
            for (int j = 0; j < m; j++) {
                if (houses[j][0] > clients[i][0] && houses[j][1] <= clients[i][1]) {
                    adj[i].add(j);
                }
            }
        }

        int[] match = new int[m];
```

```
        Arrays.fill(match, -1);
        int result = 0;
        for (int i = 0; i < n; i++) {
            boolean[] visited = new boolean[m];
            if (canMatch(i, adj, match, visited)) {
                result++;
            }
        }
        System.out.println(result);
        sc.close();
    }
}
```

---

6. Solution





- The problem effectively models a real-world resource allocation scenario using graph theory.
- The implementation successfully handles the provided constraints within the time limits.